Software Engineering 2 Labor
Bachelor Informatik 4. Semester
Intermediate Hand-In
Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali

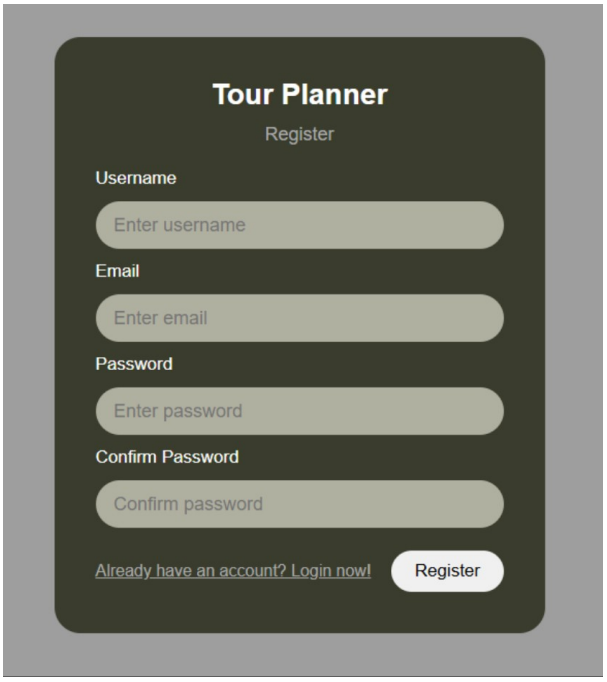
Tour Planner – Protokoll

UX Design & User Flow

The UI follows a dark-themed design with muted green accents (#7a9e6e) and rounded card-based layouts. The design prioritizes clarity and ease of use. The following sections describe each screen and the intended user flow.

Register Page

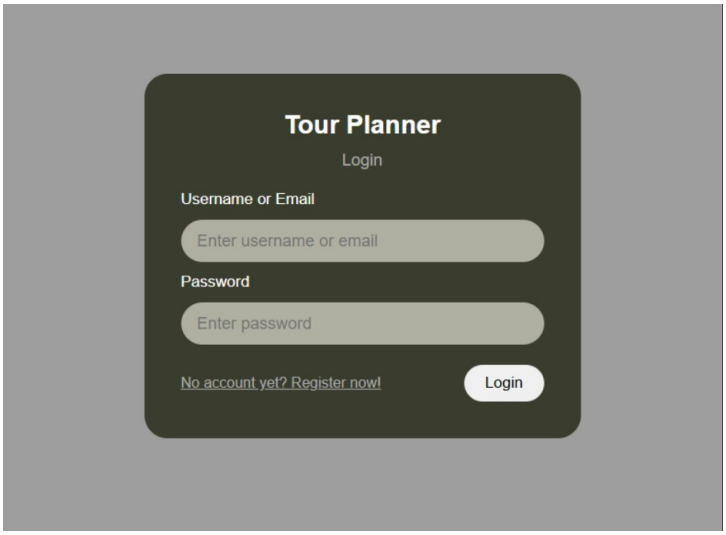
New users are greeted with a registration form upon first visit. The form collects a username, email address, password, and password confirmation. Client-side validation ensures all fields are filled and both passwords match before the request is sent to the backend. A link at the bottom navigates to the login page for returning users.



Login Page

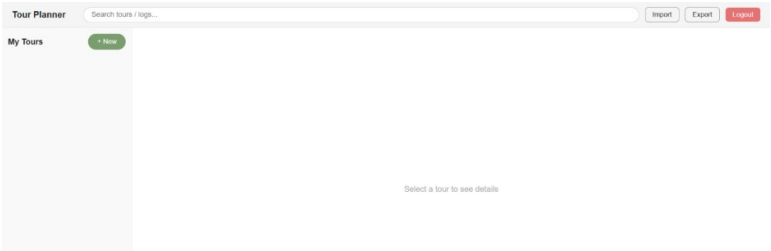
Returning users log in using either their username or email address along with their password. Upon successful authentication, the backend returns a JWT token which is stored in localStorage and automatically attached to all subsequent API requests via an HTTP

interceptor. The user is then redirected to the dashboard. Invalid credentials display an error message without revealing which field was incorrect.



Dashboard — Empty State

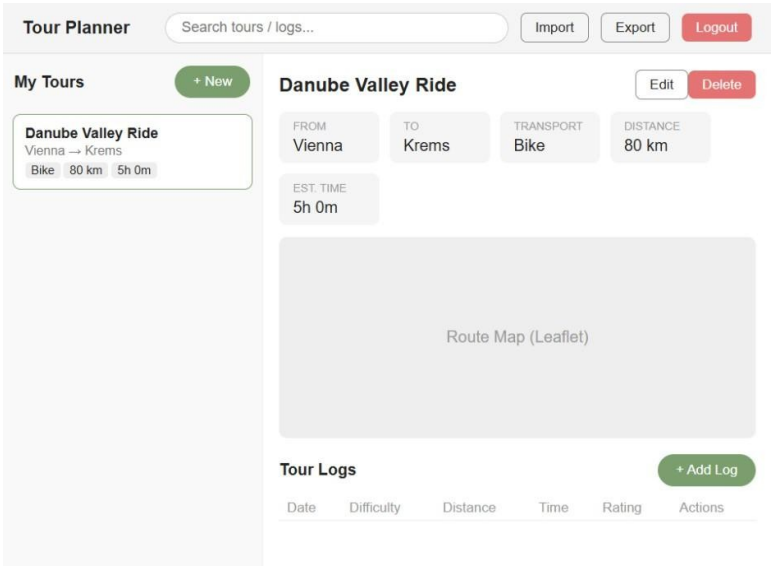
After login, users land on the main dashboard. If no tours have been created yet, the right panel displays a message prompting the user to select a tour, and the sidebar shows only the '+ New' button. The header contains a search bar, Import, Export, and Logout buttons.



Dashboard — Tour Selected

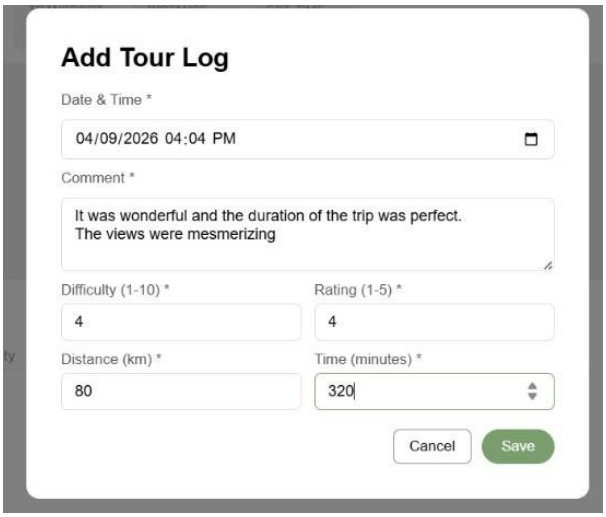
Once a tour is created and selected from the sidebar, the detail panel on the right shows all tour attributes: name, from/to locations, transport type, distance, and estimated time. Below the info cards is a map placeholder (to be replaced with a Leaflet map in the final submission). Further down, the Tour Logs section lists all logs for the selected tour in a table.

Software Engineering 2 Labor
Bachelor Informatik 4. Semester
Intermediate Hand-In
Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali



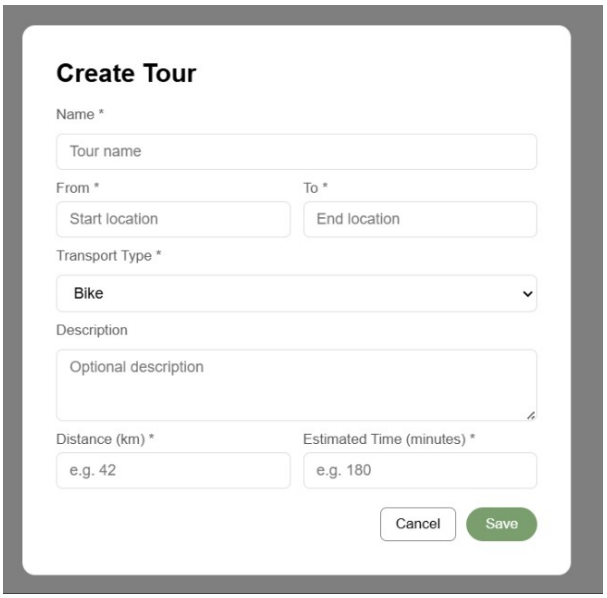
Create / Edit Tour Form

Clicking '+ New' opens a modal form where the user can create a new tour. Required fields are marked with an asterisk: name, from, to, transport type, distance (km), and estimated time (minutes). The description field is optional. Transport type is selected from a dropdown with options Bike, Hike, Running, and Vacation. A Cancel button dismisses the form and a Save button submits it to the backend.



Add / Edit Tour Log Form

Each tour can have multiple logs. Clicking '+ Add Log' opens a modal form with fields for date and time, a comment about the experience, difficulty (1-10), rating (1-5), actual distance covered, and total time in minutes. All fields are required and validated before submission. Existing logs can be edited or deleted directly from the tour log table.

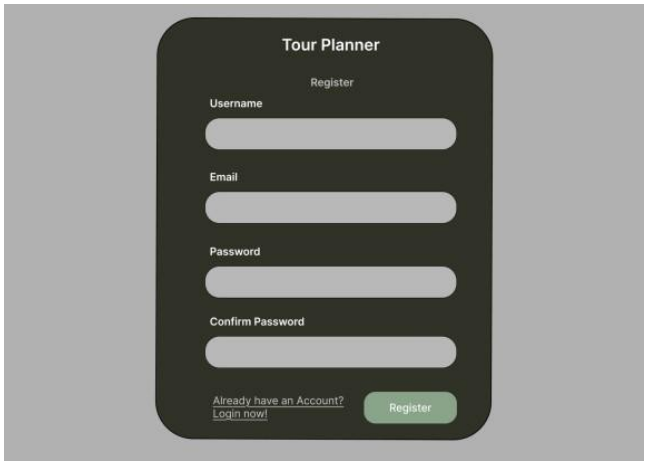


Wireframes

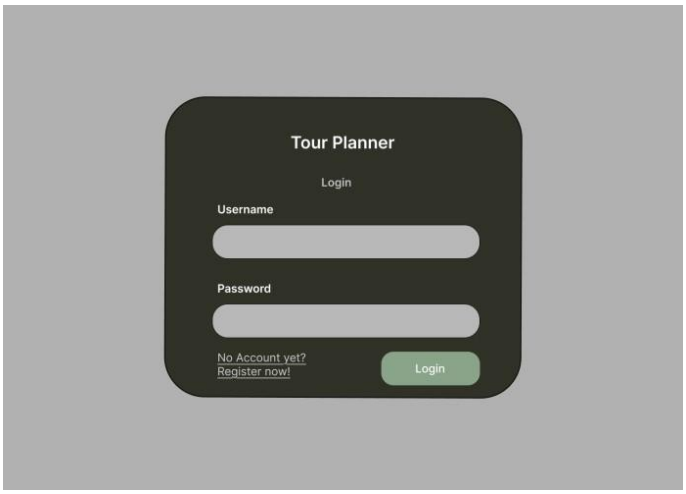
The wireframes were designed prior to implementation and served as the blueprint for the final UI. They were created in Figma and cover all primary screens of the application. The screenshots below represent the actual running application, which closely follows the original wireframe designs.

The following screens are documented below:

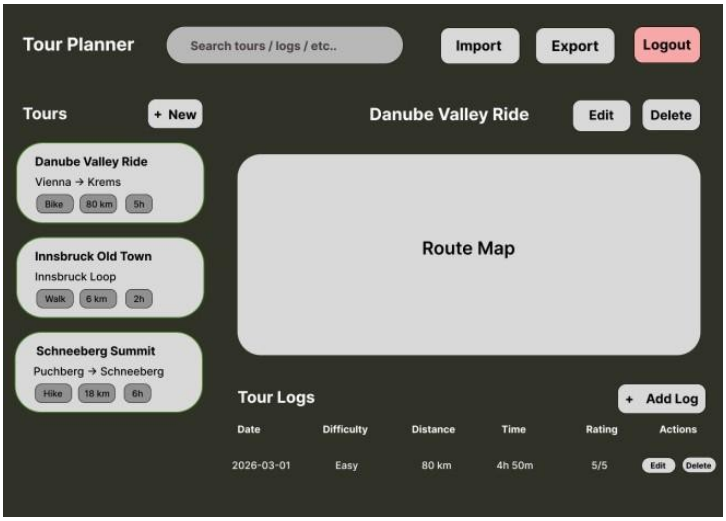
- Register — user account creation form
- Login — authentication form supporting username or email
- Empty Dashboard — initial state after login with no tours



Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali



Key Design Decisions



Dark Theme with Green Accents

A dark background color (#3a3d2e) was chosen for the auth pages and a light dashboard to provide visual contrast between the authentication flow and the main application. Muted green (#7a9e6e) is used consistently for primary action buttons (Login, Register, Save, + New, + Add Log) to guide the user toward the next action.

Modal Forms Instead of Separate Pages

Tour creation and tour log forms are implemented as modal overlays rather than separate routes. This keeps the user on the dashboard and avoids full page reloads, resulting in a faster and more fluid experience.

Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali

Sidebar + Detail Panel Layout

The dashboard uses a fixed sidebar for the tour list and a dynamic detail panel for the selected tour. This split-panel pattern is common in applications with list-detail relationships and allows the user to switch between tours without losing context.

Software Engineering 2 Labor
Bachelor Informatik 4. Semester
Intermediate Hand-In
Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali

Backend Architecture

The backend is a Spring Boot 3.2.5 REST API on Java 21, built with Maven. It is consumed by the Angular frontend over HTTP/JSON, stores data in PostgreSQL via JPA/Hibernate, and fetches route data from the external OpenRouteService API. The code uses a layered architecture where each layer only calls the one beneath it:

```
Controllers  (@RestController)  — HTTP endpoints, validation, DTO in/out, no logic
    |
Services     (@Service)         — business rules, ownership checks, calculations,
    |                             orchestration  --> OpenRouteServiceClient (external API)
Repositories (@Repository)      — Spring Data JPA, CRUD + custom queries
    |
PostgreSQL                      — tables: users, tours, tour_logs

Cross-cutting: Spring Security + JWT filter, DTO mapping, Bean Validation, Log4j2.
```

Class diagram (three JPA entities): User 1—* Tour 1—* TourLog. A Tour belongs to one User (@ManyToOne) and owns its TourLogs (@OneToMany, cascade ALL, orphanRemoval). Tour also has two derived attributes — popularity (number of logs) and childFriendliness (computed score, see Unique Feature).

Use Cases (Backend)

Actor: the authenticated User. After register/login the user receives a JWT that is required on every other request. Use cases: manage Tours (CRUD), manage Tour Logs (CRUD), upload/download a tour image, search tours and logs, view statistics, and import/export tours as JSON.

Sequence – Create Tour

The Create Tour flow is representative because it touches every layer and the external API:

```
Client -> TourController.createTour(JWT, TourRequest)
      -> [JwtAuthenticationFilter validates token]
      -> TourService: get current user, call OpenRouteServiceClient for distance/time,
                map DTO -> entity, TourRepository.save()
      <- 201 Created + TourResponse (DTO)
```

Library Decisions & Lessons Learned (Backend)

Library	Why
Spring Boot / Spring Data JPA	Standard framework; ORM removes boilerplate.
PostgreSQL	Robust open-source database, easy in Docker.
Spring Security + JJWT	Stateless JWT authentication.
Apache HttpClient 5 + Jackson	Calling and parsing the OpenRouteService API.
JUnit 5 + Mockito + H2	Fast unit tests; in-memory DB for repository tests.

Library	Why
Lombok, Log4j2, Docker	Less boilerplate, logging, reproducible setup.

Lessons learned:

- **Duration isn't a native JPA type —** we wrote a custom AttributeConverter to store it as minutes.
- **External APIs fail —** the route client degrades to fallback values so tour creation never breaks.
- **Never expose entities —** dedicated DTOs avoid leaking the password field and JSON recursion.

Design Pattern (Backend)

Adapter — OpenRouteServiceClient. It adapts the external API (coordinates, profile strings, metres/seconds) to the domain (location strings, transport type, kilometres, Duration), exposing one clean method getRouteInfo(from, to, transportType). Services depend only on this interface and never see the external format. Also used: DTO, Repository, and constructor-based Dependency Injection.

Unique Feature (Backend)

Computed child-friendliness score (0–100) per tour, calculated in Tour.getChildFriendliness() from three normalised, weighted factors: average log difficulty (30%), estimated time vs a 120-minute reference (35%), and distance vs a 10 km reference (35%). Shorter, easier and quicker tours score higher. This is combined with the resilient OpenRouteService integration that automatically fills real distance and travel time on tour creation.

Unit Testing (Backend)

The backend has 66 JUnit 5 tests. Services are tested in isolation with Mockito (repositories, the security context and the route client are mocked); repository queries run against an in-memory H2 database; request DTOs have validation tests; and JwtService is tested for token generation, validation and expiry. No external calls are made during tests, so the suite is fast and deterministic.

Software Engineering 2 Labor
Bachelor Informatik 4. Semester
Intermediate Hand-In
Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali

Tracked Time

Combined frontend and backend effort. Backend hours are estimates — adjust to your actual logged time.

Task	Time
Project setup (backend) — Spring Boot, Maven, PostgreSQL	1.5h
Project setup (frontend) — Angular CLI, folder structure, routing	1.5h
Wireframes design in Figma (Login, Register, Dashboard, Create Tour)	2h
Domain model & JPA entities (User, Tour, TourLog, Duration converter)	2h
Repositories & custom queries (Spring Data JPA)	1h
Tour & tour log services — business logic, ownership checks	3h
Security — Spring Security + JWT authentication filter	2.5h
OpenRouteService integration — client, geocoding, fallback	2.5h
Statistics, search & import/export services (backend)	2.5h
REST controllers & DTOs with validation	2h
Backend unit tests (66 JUnit 5 tests)	3h
Login component — HTML, TypeScript, styling, validation	1.5h
Register component — HTML, TypeScript, styling, validation	1h
Auth service — login, register, logout, token storage	1h
JWT interceptor — automatic token attachment to requests	0.5h
Auth guard — protecting dashboard route	0.5h
Tour service — CRUD HTTP calls	1h
Tour log service — CRUD HTTP calls	1h
Dashboard component — tour list, detail panel, MVVM structure	3h
Tour form component — create/edit reusable modal form	2h
Tour log form component — create/edit reusable modal form	1.5h
Global styling — dark theme, buttons, cards, responsive layout	2h
Docker setup — Dockerfile, Nginx config, docker-compose integration	1h
Leaflet map integration — TourMapComponent, geocoding, markers	2.5h
Search service + debounce logic wired to backend	1.5h
Import/Export functionality — file download and file upload	1.5h
Statistics service — popularity and child-friendliness display	1h
Bug fixes — modal z-index, delete button styling, welcome card	1h
Protocol writing (frontend + backend)	2h

Software Engineering 2 Labor
Bachelor Informatik 4. Semester
Intermediate Hand-In
Students:
Glen Kasa & Mohammad Parsa Farajzadeh Jalali

Task	Time
Total	**~49h**

Link to git

https://github.com/glenkishoti/tour_planner.git